

Chapel Call Graph Analyzer

A Comprehensive Tool for Parallel and Distributed Code Analysis

Chapel Code Analysis Framework

September 1, 2025

Contents

1	Introduction	3
2	Architecture Overview	3
2.1	Core Components	3
2.2	Analysis Phases	3
3	Complexity Metrics Reference	3
3.1	Traditional Software Metrics	3
3.1.1	Cyclomatic Complexity (CC)	3
3.1.2	Algorithmic Complexity	4
3.2	Maintainability Metrics	4
3.2.1	Halstead Complexity Measures	4
3.2.2	Maintainability Index (MI)	5
3.2.3	Lines of Code Metrics	5
3.3	Coupling and Cohesion Metrics	5
3.3.1	Fan-in and Fan-out	5
4	Chapel-Specific Metrics	6
4.1	Parallelism Analysis	6
4.1.1	Parallel Complexity Classification	6
4.1.2	Parallel Fraction (Amdahl's Law)	6
4.2	Data Locality Metrics	6
4.2.1	Cache Efficiency Estimation	6
4.2.2	NUMA Awareness	6
4.3	Scalability Metrics	7
4.3.1	Load Balancing Factor	7
4.3.2	Communication-to-Computation Ratio	7
4.4	GPU-Specific Metrics	7
4.4.1	GPU Occupancy Estimation	7
4.4.2	Memory Coalescing	7
4.4.3	Thread Divergence	8
5	Configuration Options	8
5.1	Basic Configuration	8
5.2	Analysis Configuration	8

6	Output Format	8
6.1	PikChr Diagram Elements	8
6.1.1	Node Types	8
6.1.2	Edge Types	9
6.1.3	Metric Labels	9
6.2	Complexity Report Structure	9
7	Usage Examples	9
7.1	Basic Analysis	9
7.2	Advanced Analysis	10
7.3	Specialized Analysis	10
8	Interpretation Guidelines	10
8.1	Performance Optimization	10
8.1.1	Identifying Bottlenecks	10
8.1.2	Scalability Assessment	10
8.2	Maintainability Assessment	11
8.2.1	Code Quality Indicators	11
8.2.2	Refactoring Recommendations	11
9	Technical Implementation	11
9.1	Regular Expression Patterns	11
9.2	Complexity Calculation Algorithms	11
9.2.1	Cyclomatic Complexity	11
9.2.2	Algorithmic Complexity Estimation	12
10	Limitations and Future Work	12
10.1	Current Limitations	12
10.2	Future Enhancements	12
11	Conclusion	12
12	References	13

1 Introduction

The Chapel Call Graph Analyzer is a sophisticated static analysis tool designed specifically for Chapel programming language projects. Chapel, being a parallel and distributed programming language, presents unique challenges in code analysis that require specialized metrics beyond traditional software engineering measures.

This tool generates visual call graphs using PikChr diagrams while computing comprehensive complexity metrics tailored for parallel, distributed, and GPU-accelerated code. The analyzer provides insights into code maintainability, scalability, and performance characteristics essential for high-performance computing applications.

2 Architecture Overview

2.1 Core Components

The analyzer consists of several key components:

- **Parser Engine:** Analyzes Chapel source files using regular expressions to identify functions, types, and language constructs
- **Call Graph Builder:** Constructs directed graphs representing function call relationships
- **Complexity Analyzer:** Computes multiple complexity metrics for each function
- **Visualization Generator:** Creates PikChr diagrams with embedded metrics
- **Report Generator:** Produces detailed textual analysis reports

2.2 Analysis Phases

The analysis process occurs in four distinct phases:

1. **Discovery Phase:** Identifies all Chapel source files and parses function/type definitions
2. **Call Analysis Phase:** Maps function call relationships across the codebase
3. **Complexity Analysis Phase:** Computes detailed metrics for each function
4. **Fan Metrics Calculation:** Computes coupling metrics based on call relationships

3 Complexity Metrics Reference

3.1 Traditional Software Metrics

3.1.1 Cyclomatic Complexity (CC)

Definition: McCabe's cyclomatic complexity measures the number of linearly independent paths through a program's source code.

Formula: $CC = E - N + 2P$ where:

- E = number of edges in the control flow graph
- N = number of nodes in the control flow graph
- P = number of connected components

Implementation: The analyzer counts decision points including:

- Conditional statements (`if`, `else`, `select`, `when`)
- Loop constructs (`for`, `while`, `do`)
- Boolean operators (`&&`, `||`)
- Chapel-specific constructs (`when`, `otherwise`)

Interpretation:

- $CC \leq 10$: Low complexity, easy to test and maintain
- $10 < CC \leq 20$: Moderate complexity, acceptable
- $CC > 20$: High complexity, difficult to maintain and test

3.1.2 Algorithmic Complexity

Definition: Estimates the Big O notation complexity of algorithms based on static code analysis patterns.

Detection Heuristics:

- Nested loop analysis for polynomial complexity
- Recursive function detection for exponential complexity
- Matrix operation patterns for cubic complexity
- Parallel iteration patterns for $O(n/p)$ complexity

Special Cases for Chapel:

- `forall` loops: $O(n \cdot p^{-1})$ where p is processor count
- Domain operations: Complexity based on domain dimensionality
- Distributed operations: Additional communication overhead factors

3.2 Maintainability Metrics

3.2.1 Halstead Complexity Measures

Definition: Halstead metrics quantify software complexity based on the count of operators and operands in the source code.

Core Measures:

$$n_1 = \text{number of distinct operators} \quad (1)$$

$$n_2 = \text{number of distinct operands} \quad (2)$$

$$N_1 = \text{total number of operators} \quad (3)$$

$$N_2 = \text{total number of operands} \quad (4)$$

Derived Measures:

$$\text{Vocabulary} = n_1 + n_2 \quad (5)$$

$$\text{Length} = N_1 + N_2 \quad (6)$$

$$\text{Volume} = \text{Length} \times \log_2(\text{Vocabulary}) \quad (7)$$

$$\text{Difficulty} = \frac{n_1}{2} \times \frac{N_2}{n_2} \quad (8)$$

3.2.2 Maintainability Index (MI)

Definition: A composite metric that combines multiple complexity measures to provide an overall maintainability score.

Formula:

$$MI = \max(0, 171 - 5.2 \times \ln(HV) - 0.23 \times CC - 16.2 \times \ln(LOC)) \quad (9)$$

where:

- HV = Halstead Volume
- CC = Cyclomatic Complexity
- LOC = Lines of Code

Interpretation:

- $MI > 85$: Highly maintainable
- $65 \leq MI \leq 85$: Moderately maintainable
- $MI < 65$: Difficult to maintain

3.2.3 Lines of Code Metrics

Effective Lines of Code (ELOC): Non-blank, non-comment lines that contain executable code.

Comment Density: $CD = \frac{\text{Comment Lines}}{\text{Total Lines}} \times 100\%$

Code Density: $CoD = \frac{\text{ELOC}}{\text{Total Lines}} \times 100\%$

3.3 Coupling and Cohesion Metrics

3.3.1 Fan-in and Fan-out

Fan-in: The number of functions that call a given function.

$$\text{Fan-in}(f) = |\{g : g \text{ calls } f\}| \quad (10)$$

Fan-out: The number of functions called by a given function.

$$\text{Fan-out}(f) = |\{g : f \text{ calls } g\}| \quad (11)$$

Interpretation:

- High Fan-in: Function is heavily used (potential bottleneck or critical component)
- High Fan-out: Function is highly coupled (difficult to maintain)
- Optimal: Low Fan-out, moderate Fan-in for utility functions

4 Chapel-Specific Metrics

4.1 Parallelism Analysis

4.1.1 Parallel Complexity Classification

The analyzer categorizes functions based on their parallelism patterns:

- **Sequential:** No parallel constructs detected
- **Data Parallel:** Uses `forall` loops for data parallelism
- **Task Parallel:** Uses `coforall`, `begin` for task parallelism
- **Structured Parallel:** Uses `cobegin` for structured parallelism
- **Synchronized:** Uses `sync`, `single` variables
- **GPU Parallel:** Contains GPU kernels or annotations
- **Distributed:** Uses locale-aware constructs

4.1.2 Parallel Fraction (Amdahl's Law)

Definition: Estimates the fraction of code that can be parallelized, critical for predicting speedup limits.

Formula: Parallel Fraction = $\frac{\text{Operations in Parallel Blocks}}{\text{Total Operations}}$

Amdahl's Law Application:

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{N}} \quad (12)$$

where P is the parallel fraction and N is the number of processors.

4.2 Data Locality Metrics

4.2.1 Cache Efficiency Estimation

Definition: Measures the likelihood of efficient cache usage based on memory access patterns.

Formula:

$$\text{Cache Efficiency} = \frac{\text{Sequential Accesses}}{\text{Total Array Accesses}} \quad (13)$$

Access Pattern Classification:

- **Sequential:** `array[i]`, `array[i+1]`, ...
- **Strided:** `array[i::stride]`
- **Random:** Non-predictable access patterns

4.2.2 NUMA Awareness

Definition: Measures how well code is structured for Non-Uniform Memory Access architectures.

Indicators:

- Use of `here` keyword for local operations
- Explicit `Locales` management
- `on` statements for controlled remote access

4.3 Scalability Metrics

4.3.1 Load Balancing Factor

Definition: Estimates how evenly work is distributed across processing elements.

Estimation Heuristics:

- cyclic distribution: 0.9 (excellent balance)
- block distribution: 0.8 (good balance)
- Manual distribution: 0.6 (potentially unbalanced)

4.3.2 Communication-to-Computation Ratio

Definition: Measures the overhead of communication relative to computation in distributed code.

Formula:

$$\text{Comm/Comp Ratio} = \frac{\text{Communication Operations}}{\text{Computation Operations}} \quad (14)$$

Communication Operations:

- Remote memory accesses (`on` statements)
- Synchronization points (`sync`, `barrier`)
- Inter-locale data transfers

Interpretation:

- Ratio < 0.3: Good scalability potential
- Ratio 0.3 – 0.7: Moderate scalability
- Ratio > 0.7: Communication-bound, poor scalability

4.4 GPU-Specific Metrics

4.4.1 GPU Occupancy Estimation

Definition: Estimates the percentage of GPU resources utilized by a kernel.

Simplified Formula:

$$\text{Occupancy} = \min \left(1.0, \frac{1}{\text{Register Usage Factor}} \right) \quad (15)$$

Factors Affecting Occupancy:

- Number of registers per thread
- Shared memory usage
- Block size configuration

4.4.2 Memory Coalescing

Definition: Measures the efficiency of memory access patterns for GPU architectures.

Coalesced Access Patterns:

- Consecutive threads access consecutive memory locations
- Use of `@blockIdx` and `@threadIdx` for structured access
- Aligned memory accesses

4.4.3 Thread Divergence

Definition: Identifies conditional code that may cause threads within a warp to follow different execution paths.

Detection Patterns:

- Conditional statements based on thread indices
- `if` statements within GPU kernels
- `select/when` constructs in parallel regions

5 Configuration Options

5.1 Basic Configuration

Option	Default	Description
<code>--inputDir</code>	<code>.</code>	Directory containing Chapel source files
<code>--outputFile</code>	<code>call_graph.pikchr</code>	Output file for PikChr diagram
<code>--maxDepth</code>	<code>3</code>	Maximum call depth to analyze
<code>--includeBuiltins</code>	<code>false</code>	Include built-in functions in analysis

Table 1: Basic Configuration Options

5.2 Analysis Configuration

Option	Default	Description
<code>--analyzeComplexity</code>	<code>true</code>	Enable complexity metric computation
<code>--complexityMetrics</code>	<code>true</code>	Show metrics in PikChr diagram
<code>--showParallelism</code>	<code>true</code>	Analyze parallel constructs
<code>--includeGPUkernels</code>	<code>true</code>	Include GPU-specific analysis
<code>--distributedAnalysis</code>	<code>true</code>	Analyze distributed operations

Table 2: Analysis Configuration Options

6 Output Format

6.1 PikChr Diagram Elements

6.1.1 Node Types

- **Gray boxes:** Sequential functions
- **Light green boxes:** Parallel functions (`[PAR]`)
- **Yellow boxes:** GPU kernels (`[GPU]`)
- **Light blue boxes:** Distributed functions (`[DIST]`)

6.1.2 Edge Types

- **Standard arrows:** Sequential function calls
- **Green thick arrows:** Parallel function calls
- **Gold thick arrows:** GPU kernel invocations
- **Blue thick arrows:** Distributed function calls

6.1.3 Metric Labels

Each function node displays relevant metrics:

```

1 functionName
2 [PAR]
3 CC:8 MI:72
4 O(n^2) F:2/5
5 C/C:15%
```

Listing 1: Example Node Label

Where:

- **CC:** Cyclomatic Complexity
- **MI:** Maintainability Index
- **F:** Fan-in/Fan-out ratio
- **C/C:** Communication/Computation ratio

6.2 Complexity Report Structure

The generated text report includes:

1. **Executive Summary:** High-level project metrics
2. **Function-by-Function Analysis:** Detailed metrics for each function
3. **Complexity Distribution:** Statistical analysis of complexity patterns
4. **Scalability Assessment:** Parallel and distributed performance indicators
5. **Maintainability Recommendations:** Suggestions for code improvement

7 Usage Examples

7.1 Basic Analysis

```

1 # Compile the analyzer
2 chpl chapel_analyzer.chpl -o chapel_analyzer
3
4 # Analyze current directory with default settings
5 ./chapel_analyzer
6
7 # Analyze specific project directory
8 ./chapel_analyzer --inputDir=my_chapel_project --maxDepth=4
```

Listing 2: Basic Project Analysis

7.2 Advanced Analysis

```

1 # Full analysis with all metrics enabled
2 ./chapel_analyzer --inputDir=hpc_project \
3     --maxDepth=5 \
4     --analyzeComplexity \
5     --complexityMetrics \
6     --showParallelism \
7     --includeGPUkernels \
8     --distributedAnalysis \
9     --outputFile=full_analysis.pikchr

```

Listing 3: Comprehensive Analysis

7.3 Specialized Analysis

```

1 # Focus on GPU kernel analysis
2 ./chapel_analyzer --includeGPUkernels \
3     --showParallelism \
4     --maxDepth=2 \
5     --outputFile=gpu_analysis.pikchr
6
7 # Maintainability-focused analysis
8 ./chapel_analyzer --analyzeComplexity \
9     --complexityMetrics \
10    --showParallelism=false \
11    --includeGPUkernels=false

```

Listing 4: GPU-Focused Analysis

8 Interpretation Guidelines

8.1 Performance Optimization

8.1.1 Identifying Bottlenecks

- **High Fan-in functions:** Potential performance bottlenecks
- **High Communication/Computation ratio:** Scalability limitations
- **Low GPU occupancy:** Underutilized GPU resources
- **Poor cache efficiency:** Memory bandwidth limitations

8.1.2 Scalability Assessment

- **Parallel fraction** > 0.9: Excellent scalability potential
- **Load balance factor** > 0.8: Well-distributed workload
- **Low thread divergence:** Efficient GPU execution
- **High coalesced access:** Optimal memory bandwidth usage

8.2 Maintainability Assessment

8.2.1 Code Quality Indicators

- **Maintainability Index** > 85: High-quality, maintainable code
- **Low cyclomatic complexity**: Easy to understand and test
- **Balanced fan-out** (< 7): Well-modularized design
- **Adequate comment density** (> 15%): Well-documented code

8.2.2 Refactoring Recommendations

- **CC** > 15: Consider breaking into smaller functions
- **Fan-out** > 10: Reduce dependencies, improve modularity
- **MI** < 65: Requires significant refactoring
- **High communication ratio**: Optimize data locality

9 Technical Implementation

9.1 Regular Expression Patterns

The analyzer uses sophisticated regular expressions to identify Chapel language constructs:

```

1 // Function definitions
2 var funcDefRegex = new regex(r"proc\s+(\w+)\s*\([^~]*\) (?:\s*:\s*(\w+))?");
3
4 // Parallel constructs
5 var parallelRegex = new regex(r"\b(forall|coforall|begin|cobegin|sync|single)\b");
6
7 // GPU annotations
8 var gpuRegex = new regex(r"\b(@gpu|@assertOnGpu|@blockSize|@cuda)\b");
9
10 // Distributed operations
11 var distributedRegex = new regex(r"\b(on\s+\w+|Locales|here|numLocales)\b");

```

Listing 5: Key Regular Expressions

9.2 Complexity Calculation Algorithms

9.2.1 Cyclomatic Complexity

```

1 proc calculateCyclomaticComplexity(code: string): int {
2     var complexity = 1; // Base path
3
4     // Count decision points
5     complexity += conditionalRegex.findAll(code).size;
6     complexity += loopRegex.findAll(code).size;
7     complexity += code.count("&&");
8     complexity += code.count("||");
9     complexity += code.count("when");
10    complexity += code.count("otherwise");
11
12    return complexity;
13 }

```

Listing 6: Cyclomatic Complexity Calculation

9.2.2 Algorithmic Complexity Estimation

The analyzer uses pattern matching to estimate algorithmic complexity:

```

1 proc estimateAlgorithmicComplexity(code: string): string {
2   var maxNesting = analyzeLoopNesting(code);
3   var hasRecursion = detectRecursion(code);
4   var hasDomainIteration = detectDomainOps(code);
5
6   if hasRecursion {
7     return if hasMemoization(code) then "O(n)" else "O(2^n)";
8   } else if maxNesting >= 3 {
9     return "O(n^" + maxNesting + ")";
10  } else if maxNesting == 2 {
11    return if hasMatrixOps(code) then "O(n^3)" else "O(n^2)";
12  } else if maxNesting == 1 {
13    return if hasDomainIteration then "O(n*p)" else "O(n)";
14  } else {
15    return "O(1)";
16  }
17 }
```

Listing 7: Algorithmic Complexity Estimation

10 Limitations and Future Work

10.1 Current Limitations

- **Static Analysis Only:** Cannot account for runtime behavior
- **Heuristic-Based:** Complexity estimations are approximate
- **Limited Context:** Function analysis is performed in isolation
- **Regex Parsing:** May miss complex language constructs

10.2 Future Enhancements

- **AST-Based Parsing:** More accurate code analysis
- **Dynamic Profiling Integration:** Combine static and runtime metrics
- **Performance Prediction:** Model execution time and resource usage
- **Optimization Suggestions:** Automated refactoring recommendations
- **Dependency Analysis:** Inter-module dependency tracking
- **Memory Usage Estimation:** Detailed memory footprint analysis

11 Conclusion

The Chapel Call Graph Analyzer provides comprehensive insights into the structure, complexity, and performance characteristics of Chapel applications. By combining traditional software metrics with specialized measures for parallel, distributed, and GPU computing, the tool enables developers to:

- Identify performance bottlenecks and scalability limitations

- Assess code maintainability and quality
- Optimize parallel and distributed algorithms
- Make informed architectural decisions for HPC applications

The visual PikChr diagrams and detailed reports serve as valuable documentation for understanding code structure and guiding optimization efforts in high-performance computing environments.

12 References

1. McCabe, T.J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308-320.
2. Halstead, M.H. (1977). *Elements of Software Science*. Elsevier North-Holland.
3. Amdahl, G.M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483-485.
4. Chapel Language Specification. <https://chapel-lang.org/docs/>
5. PikChr Documentation. <https://pikchr.org/home/doc/trunk/homepage.md>